Project Proposal: Time-Aware Next Song Prediction with a Transformer

Team: Fatima Wasim and Sanjana Reddy

Background

Music streaming services such as Spotify recommend the next song based on what a listener has played before. A listening history is a time sequence: a list of (song, artist, timestamp) events, unevenly spaced, often thousands of events long.

Transformers became the standard model for this kind of sequence. SASRec applied self-attention to predict a user's next item, and later models added time awareness, such as TiSASRec, which uses the gaps between events. The current state of the art includes HSTU, a transformer architecture from Meta (ICML 2024). HSTU changes the attention mechanism and adds attention biases based on both position and the time between events. The paper reports clear gains over SASRec on MovieLens and Amazon Books, and Meta has deployed it at scale.

Significance

Next song prediction drives features like autoplay and personalized radio, where a better prediction directly affects what millions of people hear. Music is also harder than the movie and shopping data HSTU was tested on. People replay the same songs constantly, listen in sessions, and follow daily routines, so when someone listens matters as much as what they played before. To our knowledge, HSTU has not been evaluated on a public music listening dataset. Testing whether its time-aware design helps more on music than on movies is a new, useful result.

Problem Statement

Given a listener's chronologically ordered history of (track, artist, timestamp) events and the time of the request, predict the next track they will play, as a ranked list over all tracks in the catalog.

We will answer three questions:

1. Does HSTU, adapted to music, predict the next track better than older transformers (SASRec, BERT4Rec) and simple baselines?
2. Does HSTU's time-aware attention matter more for music than for movies? On MovieLens-1M, switching it off barely changes accuracy.
3. Do our music specific additions, an artist feature and long-history tuning, improve on vanilla HSTU, and do they help with replays, new discoveries, or both?

Dataset

Both datasets are public and free for research use.

Dataset	Role	Size (approx.)	Source
Last.fm 1K [5]	Our problem: music listening	19M events, 992 users	Zenodo
MovieLens-1M [6]	Reproduce the HSTU paper's results	1M ratings, 6,040 users	GroupLens

Last.fm 1K contains each user's full listening history up to May 2009, with user ID, timestamp, artist and track. It is distributed with Last.fm's permission for non-commercial use, and we will cite it as required.

Metrics for validation and verification

For each test event, the model ranks every track in the catalog (full ranking, no sampled negatives).

- **HR@10 (hit rate):** share of test events where the true next track is in the top 10.
- **NDCG@10 (primary):** like HR@10, but rewards ranking the true track higher.
- **Catalog coverage:** share of all tracks that appear in at least one top-10 list, a check against only recommending popular songs.
- **Training cost:** time per epoch and total GPU hours.

Validation during training. We log training loss and validation HR@10 after every epoch, use early stopping when validation stops improving for 5 epochs, and plot these curves for every model.

Verification against published results. We reproduce HSTU and SASRec on MovieLens-1M with the paper's protocol before touching music data. Our targets:

Model on MovieLens-1M	Published HR@10 / NDCG@10
HSTU	0.3097 / 0.1720 (paper [3])
HSTU-large	0.3294 / 0.1893 (paper [3])

On Last.fm 1K, where no HSTU results are published, our target is to beat vanilla HSTU, SASRec, BERT4Rec, a most-popular baseline and a "recently played" baseline under identical settings, averaged over 3 random seeds. We will report results separately for replays and for songs the user has never heard.

Deep learning framework

- **Framework:** PyTorch, with PyTorch Lightning for the training loop (per-epoch logging, checkpoints, early stopping).
- **Model library:** RecTools [4], an open-source (Apache 2.0) library that implements HSTU [3], SASRec [1] and BERT4Rec [8] on top of PyTorch with the same training loop, so all models are compared fairly.

- **Data processing:** python libraries
- **Compute:** UMD's Zaratan cluster.
- **Code:** one GitHub repository with pinned dependencies, fixed random seeds, and a single script that reproduces every reported table.

Training starts from the published RecTools HSTU settings: sampled-softmax loss with 128 negatives, learning rate 0.001, batch size 128 and up to 100 epochs. We then tune sequence length (50, 200, 500 events) and model size (2 versus 8 layers).

Existing implementation and our adaptation

We follow the RecTools HSTU implementation, which reproduces the architecture from the HSTU paper (official code: meta-recsys/generative-recommenders), and cite both. We adapt it to our problem as follows:

From the existing implementation	Our adaptation
HSTU trained on movie and book ratings	Applied to 19M music listening events, with our own data pipeline
Item IDs only	Artist added as an item feature (movie genre when we run on MovieLens-1M)
Default sequence length and size	Tuned for long histories: 50 to 500 events, 2 versus 8 layers
Leave-one-out evaluation	Time-based split, plus separate results for replays and new songs
Time-aware attention evaluated on movies	Ablation testing whether time awareness matters more for music

References

1. W.-C. Kang, J. McAuley, "Self-Attentive Sequential Recommendation," ICDM, 2018.
2. J. Li, Y. Wang, J. McAuley, "Time Interval Aware Self-Attention for Sequential Recommendation," WSDM, 2020.
3. J. Zhai et al., "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations," ICML, 2024. arXiv:2402.17152.
4. MTS, "RecTools: Transformers HSTU tutorial," RecTools documentation, rectools.readthedocs.io, accessed Oct 2026.
5. O. Celma, *Music Recommendation and Discovery*, Springer, 2010. Dataset: Last.fm 1K on Zenodo, distributed with permission of Last.fm.
6. F. M. Harper, J. A. Konstan, "The MovieLens Datasets: History and Context," ACM TiiS, 2015.
7. W. Krichene, S. Rendle, "On Sampled Metrics for Item Recommendation," KDD, 2020.
8. F. Sun et al., "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," CIKM, 2019.